How it works (the viva sheet)

1. One request, start to finish

```
browser → public/index.php → core/bootstrap.php → Core\App → routes/web.php → Core\Router → a controller → a view
```

Step	File	What it does
1	`public/index.php`	The only PHP file the browser can reach. A real file (CSS, JS, image) is sent as it is.
2	`core/bootstrap.php`	Reads `.env`, sets the timezone (`country('timezone')`), starts the session (cookie: `httponly`, `SameSite=Lax`, strict mode), and decides how errors show (`APP_DEBUG`).
3	`core/App.php`	Loads `routes/web.php`, cleans the address (`/student/` → `/student`), hands it to the Router.
4	`routes/web.php`	The list of addresses: `$router->get('/student/academic', [AcademicController::class, 'student'])`. A module switched off in `features.php` has no routes at all.
5	`core/Router.php`	Finds the address (unknown → `errors/error.php`, 404). **Every POST passes `checkCsrf()` here first**: the one place CSRF is checked.
6	`app/Controllers/…`	Decides: `requireRole('student')`, reads the form (`postText()` …), asks the models, picks the view.
7	`app/Models/…`	The only place with SQL (prepared statements, `:name` placeholders).
8	`app/Views/{role}/…`	Only shows. Every value goes through `e()`.

2. Folders

Folder	Holds
`core/`	The small framework: App, Router, Controller (base), Model (base: `transaction()`), Database (PDO), Mailer, Upload, Validator, `helpers.php`
`app/Controllers/`	One per feature (Notice, Calendar, People, Academic…)
`app/Models/`	SQL, one per table group (`UserModel`, `NoticeModel`, `Membership` …)
`app/Services/`	Work that is neither SQL nor a page: shaping data (`RecordBook`, `AchievementTimeline`, `Programmes`), the PDF (`CertificatePdf`)
`app/Mock/`	Example data for the pages whose backend isn't built yet
`app/Views/{role}/`	One page file per role and page (`student/dashboard.php`)
`app/Views/components/`	Shared pieces (`_dropdown.php`, `_pop_up.php` …); differences come from a `$variant`, not copies
`app/Views/layouts/`	`page_top.php` (head, sidebar) and `page_bottom.php` (scripts every page needs)
`config/`	Settings and rules: `permissions.php` (who may do what), one file per feature
`config/white_label/`	Everything another school changes: `school.php`, `wording.php`, `features.php`, `country.php`, `academic.php`
`public/assets/css/`	`global.css` (colour tokens, resets, the app shell), `components/` (one file per component), `layout/` (page rows: `l-…-grid`)

Folder	Holds
`public/assets/js/components/`	One file per component
`database/`	`schema.sql`, `seed.sql`, `updates/` (one numbered `.sql` file per change; `update.php` runs the new ones and remembers them in the `database_updates` table)
`bin/check.sh`	The rule check (0 errors before a hand-in)

3. How a page is made

A view sets four lists, then includes the top and bottom of the page:

```
$tabTitle = 'Notices';
$styles   = ['notice-card', 'dropdown'];       // public/assets/css/components/{name}.css
$scripts  = ['dropdown'];                      // public/assets/js/components/{name}.js
$grids    = ['notice'];                        // public/assets/css/layout/{name}-grid.css
require dirname(__DIR__) . '/layouts/page_top.php';
…  the components, e.g.  require dirname(__DIR__) . '/components/_notice_card.php';
require dirname(__DIR__) . '/layouts/page_bottom.php';
```

- A page loads only the CSS and JS it lists, so nothing unused is downloaded.
- Component = its PHP + its own CSS + its own JS. `$variant` gives the per-use differences.
- Page rows (`l-chart-calendar-grid`) only arrange: columns, gaps, screen sizes. Never a component's look.
- `Controller::page()` gives every page: `currentRole`, `currentPath`, `userName`, `csrfField`, `sessionTimeout`, and for a parent `children` + `chosenChild`.

4. Security, rule by rule

Threat	What stops it	Where
SQL injection	Prepared statements only; a column name built from array keys only from a fixed list in the code	`app/Models/`
XSS (script in a name)	`e()` on every value shown	`core/helpers.php`, every view
CSRF (another site sends our form)	A secret token in every form, checked once for every POST	`core/Router.php` → `Controller::checkCsrf()`
Wrong role	`requireRole('admin')` at the top of every action; who may do what lives in `config/permissions.php`, read with `can()`	`core/Controller.php`
Finding which accounts exist	An unknown ID, an account not activated and a wrong password get the same message; "forgot password" says the same for every ID	`AuthController`
Password guessing	Failed tries are counted; the account locks for a while (`app.login`)	`UserModel::recordFailedLogin()`
Email / code flooding	A wait between codes (`secondsUntilResend()`)	`VerificationCode`
Stolen session	`session_regenerate_id()` at sign-in, cookie `httponly` + `SameSite`, signed out after idle minutes (a warning a minute before)	`AuthController`, `bootstrap.php`, `session-timeout.js`

Threat	What stops it	Where
Strange form values (`field[]=x`)	Values only through `postText()` / `postPassword()` / `postList()` / `queryText()` : anything not plain text becomes `''`	`core/Controller.php`
Passwords	Stored with `password_hash()` , checked with `password_verify()`	`UserModel`
Error details	`APP_DEBUG=false` → people see `errors/error.php` ; the details go to `storage/logs/`	`bootstrap.php`
Background (JSON) requests	Every path answers JSON, also "signed out" and "not allowed" (`refuse()`)	`core/Controller.php`
Half-saved data	Changes that belong together run in `transaction()` : all or nothing	`core/Model.php`
Secrets	Only in `.env` (never in the code, never sent to the browser)	`.env`

5. White-label: another school changes `config/white_label/` only

Need	Helper	File
Name, logo, email, phone	`school('name')`	`school.php`
Grade / Class / Term / Student…	`word('class')` , `plural('class')` , `words('{Class} {Teacher}')`	`wording.php`
Switch a module off	`feature('notices')` (its routes, sidebar link and pages go)	`features.php`
Date format, timezone	`country('date_format')` ; JS reads `<html lang>`	`country.php`
Grades, ID formats	`config('white_label.academic…')`	`academic.php`

6. Features that run in the browser

File	Does
`pop-up.js`	Opens / closes `<dialog>` pop-ups (`data-pop-up-open="id"`); the browser gives Esc, the dark backdrop and focus
`question-pop-up.js`	"Are you sure?" questions (`askQuestion()`), also on forms with `data-question-*`
`unsaved-changes.js`	Asks before leaving a changed form (`data-warn-unsaved`) or closing a typed-in pop-up
`send-once.js`	Turns a form's buttons off while it sends, so a double click can't send it twice (also on sign-in / sign-up)
`session-timeout.js`	"Still There?" a minute before the idle sign-out
`feedback-banner.js`	The sliding banner (section 8); on example-data pages it shows what would happen
`dropdown.js`	Every styled dropdown (keyboard, multi-select, kept inside the window); a must-choose one (`$required`) stops its form while empty
`sidebar.js`	Collapses the menu (remembered in a cookie); on a phone it starts collapsed

7. Example data (mock)

Pages whose backend isn't built yet (feedback, complaints, certificates, extracurricular, achievements, term marks, dashboards' numbers) read `app/Mock/*.php`. Their forms carry `data-mock-message`: nothing is saved, the banner says what would happen. Each one's real backend is listed in `team/CHECKLIST.md`.

8. Telling the person — which tool when

One question decides it: **must the person decide something, or are we just telling them?**

The situation	Tool	How
Must **decide** — it can't be undone	Question pop-up	`askQuestion({…})` or a form with `data-question-title/-text/-button`
Must **fill in or look at** something, without leaving the page	Pop-up with content	`components/_pop_up.php`
"It worked"	Sliding banner, green, leaves after a few seconds	PHP: `$this->setFlash('notice', 'Notice pinned.')` before the redirect · JS: `showFeedbackBanner('…')`
"It didn't work" after a one-click action	Sliding banner, red, **stays until ×**	PHP: `refuse()` (it sets the 'error' flash) · JS: `showFeedbackBanner('…', 'error')`
"It didn't work" in a form	Red box inside the form	the controller reads `getFlash('error')` for its form → `.c-banner--error`
A hint or warning to read while working	Box in the page	`.c-banner--info` / `--warning`

How the flash reaches the banner: `Controller::page()` reads what is left of 'notice' / 'error' → `layouts/page_bottom.php` draws the one sliding banner of the page. A form page reads its 'error' first (while its data is built), so a form's problem is shown once, inside the form. `bin/check.sh` refuses a green box in a page and a page that draws the banner itself. (The sign-in pages are outside the signed-in frame: their messages are about the form you're about to fill, so they stay boxes there.)

9. The database in three layers

Layer	What	Tables
Structure — set up once, rarely changes	the school itself	`grades`, `classes`, `stages`, `stage_subjects`, `extracurriculars`
Assignments — who is where, per school year	changes every year; earlier years are kept	`class_students`, `class_teachers`, `subject_teachers`, `teachers_in_charge` (each row has a `year`) · club members: `extracurricular_members` (now) + `extracurricular_member_history` (every membership that ended)
Records — what happened	kept for good	`marks`, `term_comments`, `achievements`, `notices`, `events`, `audit_log` …

- The current school year = the newest row of `school_years`. Year-end (B-9) adds the next one; until then a new calendar year changes nothing on its own.
- Pages read this year's assignments through the views `current_class_students`, `current_class_teachers`, `current_subject_teachers`, `current_teachers_in_charge` and `current_teaching` (every class a teacher teaches). A view is a saved SELECT: `SELECT … FROM class_students WHERE year = (newest year)`.
- Only `App\Models\Assignments` writes the assignment tables (`placeStudent`, `setClassTeacher`, `setSubjectTeacher`, `setTeacherInCharge`, `hasRoom`).

- A class that had students in an earlier year can't be deleted: its records point to it (the database refuses it, and the Academic page says so first).
- Term marks: the class teacher types them (0–100 or AB) and presses Publish Term; students and parents see only published terms. Totals, averages, positions and the automatic awards are never stored: `App\Services\TermResults` works them out from the marks every time, so a correction changes everything at once. The school's rules (Commended 80 %, Top of Class, Top of Grade, each on / off) are in `config/white_label/rules.php`, read with `rule('…')`.
- A subject with marks is never deleted or renamed: taken out of a curriculum it is retired (`retired_on`), so old reports keep showing it under its old name.